AI-Driven Clinical Decision Support System (CDSS)

Development & Architecture Plan

Based on the System Requirements Specification (SRS)

Advanced Database Systems - Project

Table of Contents

1. Project Overview

The AI-Driven Clinical Decision Support System (CDSS) is an advanced database system designed for physicians and clinicians to validate new prescriptions during patient consultations. It leverages NoSQL Multi-Model architectures and AI to prevent adverse drug events (ADEs) by cross-referencing new prescriptions with the patient's flexible medical history and existing medications.

The system serves as a point-of-care tool for doctors. When a doctor queries a patient profile and inputs a proposed medication, the system utilizes a Graph Database model to instantly flag exact drug-drug interactions or allergen conflicts, while employing a Vector Database and LLM (RAG pipeline) to generate alternative clinical options and precise medical reasoning.

2. Current State Analysis

The project currently exists as a basic Streamlit + MongoDB patient management system with the following structure:

```
dds/
  app.py              -- Main entry, page routing
  config/
    database.py        -- MongoDB connection (local:27017, DB: medical_DDS)
  layouts/
    sidebar.py         -- Navigation (Home, Dashboard, Add Patient, View Patients)
  services/
    model.py           -- Empty
    patient_service.py -- CRUD for patients
  ui/
    add_patient.py     -- Form: name, DOB, gender, chronic conditions
    view_patient.py    -- Table view with search
    dashboard.py       -- Empty
    validation.py      -- Empty
  .vscode/
    settings.json
```

Key observations: MongoDB holds patients, medications, and counters collections. The model.py, dashboard.py, and validation.py files are empty -- the core CDSS functionality (drug interaction checking, AI clinical advisor) has not been implemented yet.

3. Requirements Gap Analysis

Below is the mapping of SRS functional requirements to current implementation status:

Requirement	Status	Gap
FR-1.1/1.2 (Doc Store)	Partial	Basic patient info exists; no medical history, active meds, or allergies embedded
FR-2.1-2.3 (Graph Store)	Missing	No Neo4j or graph traversal; no drug interaction or allergy conflict checking
FR-3.1-3.3 (Vector & LLM)	Missing	No vector DB, no embeddings, no LLM RAG pipeline
NFR-1.1 (Deterministic first)	Missing	No separation of deterministic vs probabilistic outputs in UI
NFR-1.2 (Speed < 1s)	Missing	No graph adjacency traversal implemented

Because the project currently lacks a dedicated graph database (Neo4j), the graph traversal requirement will be implemented using MongoDB edge documents -- storing drug-drug interactions and drug-allergen relationships as structured documents with source and target references, queried via MongoDB aggregation pipelines.

4. Technology Stack

The following technologies were selected after evaluating project constraints and academic context:

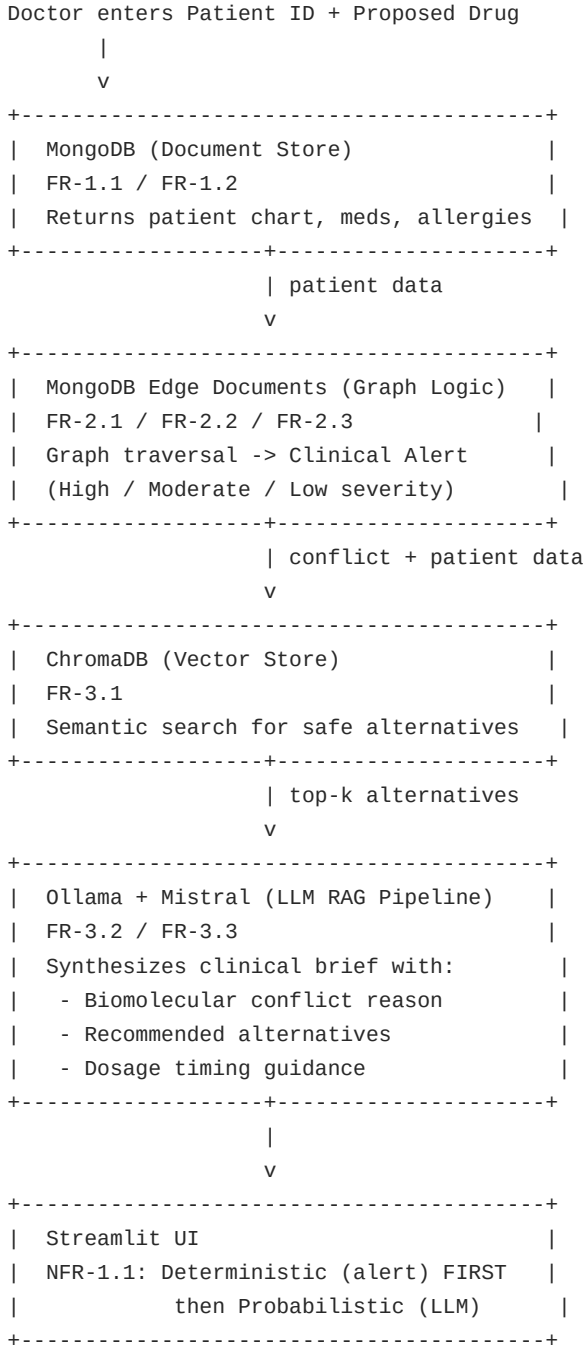
Layer	Technology	Purpose
Frontend	Streamlit	Physician UI -- dashboards, forms, alerts
Doc Store	MongoDB (local)	Patient charts, medical history, conditions
Graph Logic	MongoDB edge docs	Drug-drug and drug-allergen relationships (no Neo4j)
Vector Store	ChromaDB (local)	Semantic search of medical knowledge base
Embeddings	all-MiniLM-L6-v2	Sentence-transformers for drug/condition vectors
LLM	Ollama + Mistral	Local RAG pipeline -- no API costs, fully offline
Language	Python 3	Backend orchestration and data processing

Justification for key decisions:

- MongoDB-as-Graph: User has no Neo4j installed; edge documents in MongoDB provide an elegant compromise for an academic project while still demonstrating graph traversal concepts.
- ChromaDB: Lightweight, file-based, no server process needed. Pairs naturally with local development. MongoDB Atlas Vector Search requires cloud migration.
- Ollama + Mistral: Free, fully offline, academic-friendly. Set up with ollama pull mistral. Can be swapped to OpenAI by changing one config value.

5. System Architecture & Data Flow

End-to-end flow when a doctor validates a prescription:



Architecture notes:

- NFR-1.1 (Separation): The UI displays the deterministic graph-derived alert in a distinct colored banner BEFORE any LLM-generated content. A visual separator and label ('AI Clinical Advisor') clearly distinguish probabilistic content.
- NFR-1.2 (Speed): MongoDB indexes on drug_a, drug_b, and patient_id fields ensure sub-second lookups. For a production system, Neo4j's index-free adjacency would provide faster traversals, but for academic scale (~100 drugs), MongoDB aggregation performs well enough.

6. Implementation Roadmap

The implementation is organized into six logical phases:

Phase 1: Enhance Document Store (MongoDB)

- Define Pydantic models for Patient, Medication, Allergy, DrugInteraction, DrugNode in services/model.py
- Add drug_interactions, drug_kb, allergens collections to config/database.py
- Add service functions: get_patient_by_id(), add_medication(), add_allergy()
- Enhance ui/add_patient.py with fields for current medications, allergies, medical history
- Enhance ui/view_patient.py with full patient chart (history, conditions, active meds, allergies)

Phase 2: Graph Logic -- Drug Interaction Checking

- Create services/drug_graph_service.py -- graph traversal logic on MongoDB edge documents
- Drug interaction document schema: { drug_a, drug_b, type, severity, mechanism }
- Functions: check_drug_interaction(patient_id, proposed_drug) -> Alert object
- Match proposed drug vs patient's active medications; match vs patient's allergies
- Return structured alert: severity (high/moderate/low), conflicting items, interaction mechanism

Phase 3: Vector Store -- ChromaDB

- Create config/vector_db.py -- ChromaDB client + embedding function (sentence-transformers)
- Create services/rag_service.py with seed_knowledge_base() and vector_search_alternatives()
- Chunk medical knowledge on drug classes and alternatives into vector DB
- Semantic search: given therapeutic class and patient allergies, find safe alternatives

Phase 4: LLM -- Ollama RAG Pipeline

- Create services/llm_client.py -- Ollama API abstraction (requests to localhost:11434)
- Build structured prompt: conflict details + alternatives + patient history + system instruction
- Function: query_llm(prompt) -> clinical brief with biomolecular reason, alternatives, dosage timing
- Abstracted interface -- swapping to OpenAI requires changing only the client implementation

Phase 5: UI -- Prescription Validation Page

- Transform ui/validation.py into the core CDSS page
- Patient ID lookup with auto-fill patient chart summary
- Drug name input with autocomplete from drug knowledge base
- TOP SECTION (deterministic): Clinical Alert banner -- red/orange/green with severity and details
- BOTTOM SECTION (probabilistic): AI Clinical Advisor box with LLM-generated brief
- Clear visual separator between deterministic and probabilistic content (NFR-1.1)

Phase 6: Dashboard & Integration

- Complete ui/dashboard.py with summary stats, recent flagged interactions, quick patient lookup
- Update layouts/sidebar.py with Prescription Validation navigation item
- Wire all pages in app.py

7. File Plan: Create vs Modify

Summary of all files to be created and modified during implementation:

New Files to Create

File	Purpose
services/model.py	Pydantic models: Patient, Medication, Allergy, DrugInteraction, DrugNode
services/drug_graph_service.py	Drug interaction graph traversal on MongoDB edge documents
config/vector_db.py	ChromaDB client, embedding function, collection management
services/rag_service.py	Vector search, knowledge base seeding, RAG prompt builder
services/llm_client.py	Ollama API abstraction layer
data/seed_drugs.py	Seed ~40 common drugs with therapeutic classes and classifications
data/seed_interactions.py	Seed ~20 known drug-drug interactions with severity and mechanisms
data/seed_knowledge.py	Seed ChromaDB with chunked medical knowledge text

Existing Files to Modify

File	Purpose
config/database.py	Add drug_interactions, drug_kb, allergens collections
services/patient_service.py	Add: get_patient_by_id(), add_medication(), add_allergy()
ui/add_patient.py	Add fields: current medications, allergies, medical history notes
ui/view_patient.py	Show full patient chart with collapsible sections
ui/validation.py	Full implementation: prescription input, alert display, AI advisor
ui/dashboard.py	Full implementation: physician dashboard with stats and alerts feed
layouts/sidebar.py	Add Prescription Validation navigation option
app.py	Wire validation and dashboard pages

8. Seed Data Strategy

To make the system demonstrable, the following seed data will be loaded:

Drugs (~40 drugs)

- Analgesics: Aspirin, Acetaminophen, Ibuprofen, Naproxen, Celecoxib, Tramadol
- Anticoagulants: Warfarin, Heparin, Enoxaparin, Rivaroxaban, Apixaban
- Antidiabetics: Metformin, Insulin, Glipizide, Sitagliptin, Empagliflozin
- Cardiovascular: Lisinopril, Losartan, Metoprolol, Atorvastatin, Amlodipine
- Antibiotics: Amoxicillin, Azithromycin, Ciprofloxacin, Doxycycline, Cephalexin
- Psychiatric: Sertraline, Fluoxetine, Alprazolam, Lorazepam, Duloxetine
- GI: Omeprazole, Pantoprazole, Ondansetron, Metoclopramide
- Other: Prednisone, Furosemide, Levothyroxine, Allopurinol

Drug Interactions (~20 pairs)

Example interactions seeded with type, severity, and biomolecular mechanism:

- Warfarin + Aspirin -> HIGH: Increased bleeding risk, synergistic anticoagulation
- Warfarin + Ibuprofen -> HIGH: NSAID inhibits platelet aggregation, increases INR
- ACE Inhibitors + NSAIDs -> HIGH: Reduced antihypertensive effect, risk of renal failure
- Metformin + Contrast Dye -> MODERATE: Risk of lactic acidosis in renal impairment
- Ciprofloxacin + NSAIDs -> MODERATE: Increased risk of CNS stimulation and seizures
- Statins + Macrolide antibiotics -> HIGH: Increased risk of rhabdomyolysis
- SSRIs + MAOIs -> HIGH: Risk of serotonin syndrome
- SSRIs + NSAIDs -> MODERATE: Increased bleeding risk
- Warfarin + Amoxicillin -> MODERATE: Antibiotics alter gut flora, potentiate warfarin
- Metformin + Beta Blockers -> LOW: Beta blockers mask hypoglycemia symptoms
- Aspirin + Ibuprofen -> MODERATE: Ibuprofen may reduce aspirin's cardioprotective effect

Allergens

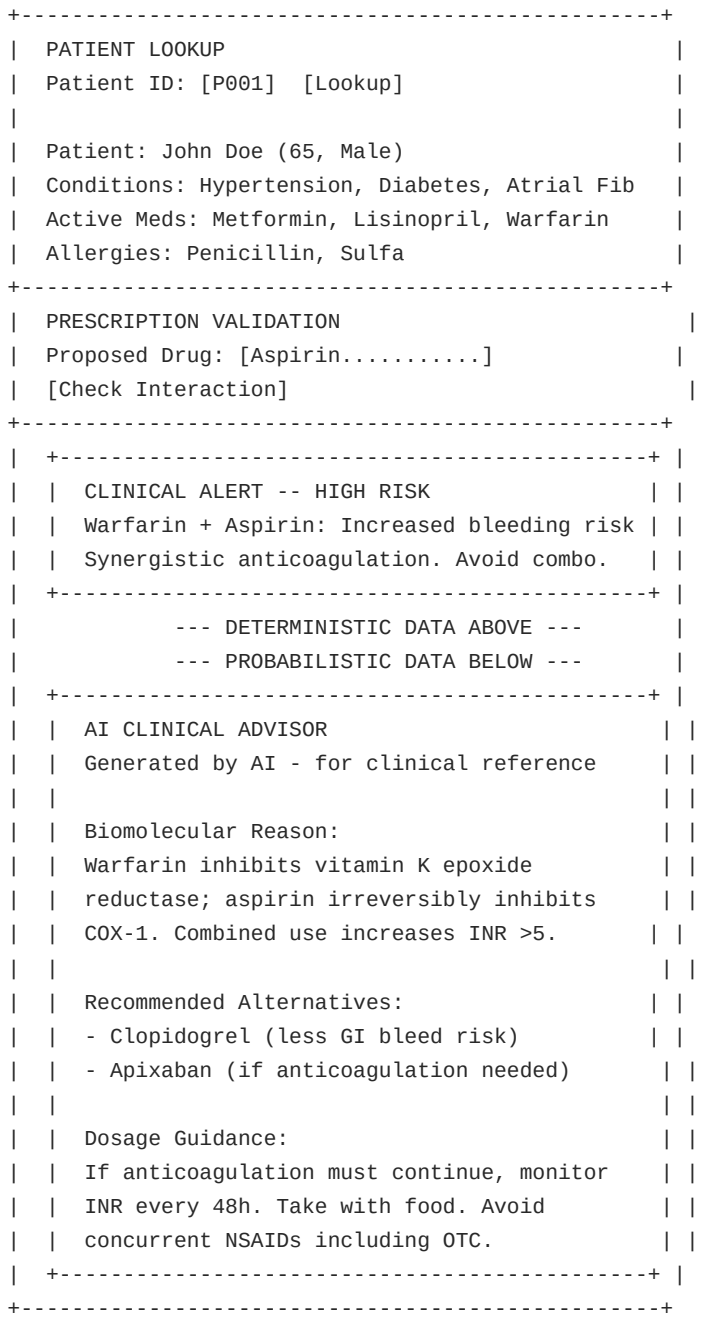
- Drug allergens: Penicillin, Sulfa, Aspirin/NSAIDs, Codeine, ACE Inhibitors, Statins
- Environmental/Other: Latex, Iodine contrast, Peanuts (for excipient checking)

Knowledge Base (Vector DB)

Chunked medical text covering therapeutic classes, mechanism of action, common alternatives, and dosing guidelines. Each chunk is 100-200 tokens, embedded with all-MiniLM-L6-v2, stored in ChromaDB with metadata (therapeutic_class, drug_name, source).

9. UI Layout for Validation Page

The prescription validation page is the core interface of the CDSS. Below is the wireframe layout:



NFR-1.1 is enforced by the clear visual separator and distinct styling between the deterministic alert (bordered, red/orange/green) and the probabilistic AI advisor (boxed, labeled 'Generated by AI').

10. Implementation Sequencing

The following order ensures each step builds on the previous one:

#	File	Action	Description
1	services/model.py	Define data models	Foundation for all data structures
2	config/database.py	Add new collections	Backend schema expansion
3	services/patient_service.py	Add lookup & med/allergy functions	Enhanced data access layer
4	services/drug_graph_service.py	Interaction checking logic	Core graph traversal engine
5	config/vector_db.py	ChromaDB setup	Vector infrastructure
6	services/llm_client.py	Ollama client	LLM integration layer
7	services/rag_service.py	RAG pipeline	Combines vectors + LLM
8	data/seed_drugs.py	Seed drug data	Populate drug knowledge base
9	data/seed_interactions.py	Seed interactions	Populate interaction graph
10	data/seed_knowledge.py	Seed vector DB	Populate medical knowledge
11	ui/add_patient.py	Enhance form	Richer patient data entry
12	ui/view_patient.py	Enhance chart view	Full patient chart display
13	ui/validation.py	Full implementation	Core CDSS page
14	ui/dashboard.py	Full implementation	Physician dashboard
15	layouts/sidebar.py	Add nav item	Navigation update
16	app.py	Wire all pages	Final integration